



UNIVERSIDAD NACIONAL DE ASUNCIÓN

INGENIERÍA MECATRÓNICA

Procesamiento Digital de Señales

Ectos para una Señal de Audio

Integrantes:

Victor Gabriel Curiel Gonzalez
Ximena Lujan Quenhan Riveros

Profesores:

Dr. Ing. Enrique Vargas
Lic. Federico Morán

Segundo Ciclo Académico, 2024

Índice general

1.	Introducción	1
2.	Descripción del problema	2
3.	Objetivos	3
3.1.	Objetivo general	3
3.2.	Objetivos específicos	3
4.	Alcance y Limitaciones	4
5.	Marco Teórico	5
5.1.	PSOC 5LP	5
5.2.	Efecto de audio	6
5.3.	Retardos	6
5.4.	Filtrado digital	7
5.5.	Modulación	9
5.6.	Efectos a ser implementados	10
5.7.	Aliasing	11
5.8.	Esquema general de la solución	13
5.9.	Filtro de reconstrucción	17
6.	Justificación y diseño de filtros seleccionados	19
6.1.	Diseño a base de bloques en el PSoC Creator	19
6.2.	Justificación de filtros seleccionados	20
6.3.	Implementación en MATLAB	20
6.4.	Implementación en PSoC	28
7.	Montaje y resultados	36
7.1.	Montaje final	36
7.2.	Resultados obtenidos	36

8.	Conclusión	40
----	----------------------	----

1. Introducción

El procesamiento digital de señales ha abierto nuevas posibilidades para modificar y mejorar las señales de audio, convirtiéndose en una herramienta indispensable en campos como la música, el cine y las aplicaciones interactivas. Al permitir la transformación controlada del sonido, el procesamiento digital de señales facilita la creación de efectos que no solo mejoran la calidad del audio, sino que también ofrecen nuevas dimensiones creativas al modificar características como el tiempo, la frecuencia y la amplitud de las señales.

En este ámbito, los conceptos de retardo y modulación desempeñan un papel central, ya que forman la base de muchos de los efectos más utilizados. La capacidad de alterar el tiempo y la amplitud de las señales de forma controlada permite generar transformaciones creativas que realzan la percepción del sonido. Los microcontroladores, como el PSoC 5LP, ofrecen una plataforma versátil para implementar estos conceptos, al combinar hardware configurable con herramientas de desarrollo que facilitan el diseño de algoritmos para la aplicación de efectos de audio.

El objetivo de este proyecto es diseñar e implementar efectos de audio como vibrato, chorus, flanger y eco, explorando sus fundamentos teóricos y llevándolos a la práctica mediante el uso del PSoC 5LP. Este dispositivo permite desarrollar algoritmos que emplean retardos y modulaciones, proporcionando una oportunidad para aplicar conceptos claves del procesamiento digital de señales en un entorno educativo.

A continuación se presenta la descripción del problema, el alcance y limitaciones. Luego un marco teórico con la descripción de todos los conceptos necesarios para poder llevar a cabo con éxito el proyecto. También se incluyen el diseño y la implementación del mismo en el PSoC Creator junto a simulaciones realizadas en MATLAB. Finalmente se presentan los resultados obtenidos.

2. Descripción del problema

El desafío principal de este proyecto radica en la implementación de efectos de audio digitales, utilizando el microcontrolador PSoC 5LP como plataforma de procesamiento de señales. Estos efectos, como vibrato, chorus, flanger y eco, requieren un manejo preciso y eficiente de la señal de entrada, asegurando que las transformaciones aplicadas no introduzcan distorsiones significativas ni comprometan la calidad del audio procesado.

La implementación de los efectos implica diseñar algoritmos que empleen retardos, modulaciones y combinaciones de señales, ajustándose a las características específicas del hardware del PSoC 5LP. Dado que cada efecto tiene requisitos únicos en términos de parámetros como frecuencia de modulación, profundidad y tiempo de retardo, es crucial optimizar su ejecución para obtener resultados consistentes y de alta calidad.

El microcontrolador PSoC 5LP se presenta como una solución adecuada gracias a sus capacidades de procesamiento digital y su versatilidad para implementar los componentes necesarios, como ADC y DAC. La metodología incluye el diseño y simulación de los algoritmos, seguido de pruebas experimentales para evaluar tanto la calidad perceptual de los efectos como su respuesta a distintas configuraciones y entornos.

3. Objetivos

3.1. Objetivo general

- Diseñar e implementar diferentes efectos para señales de audio utilizando un microcontrolador PSoC 5LP.

3.2. Objetivos específicos

- Capturar una señal analógica y realizar la conversión a digital utilizando el microcontrolador PSoC.
- Configurar y programar módulos de hardware del PSoC para procesar señales de audio.
- Diseñar e implementar filtros digitales para el procesamiento digital de audio.
- Desarrollar métodos de modulación de retardo para generar los efectos de audio.
- Convertir la señal digital procesada a formato analógico para su reproducción.
- Diseñar e implementar filtros analógicos para acondicionar las señales.

4. Alcance y Limitaciones

El proyecto se centra en la implementación de efectos de audio utilizando un microcontrolador PSoC 5LP, aprovechando sus capacidades para realizar procesamiento digital de señales (DSP). Se desarrollaron efectos como vibrato, chorus, flanger y eco, mediante el diseño e implementación de algoritmos que emplean retardos y modulaciones en el dominio del tiempo. El alcance incluye el estudio teórico de los fundamentos de procesamiento de señales, la programación de los algoritmos en el entorno PSoC Creator y la evaluación práctica de los efectos mediante pruebas auditivas y ajustes para optimizar su desempeño. Este proyecto proporciona una experiencia práctica en el diseño de sistemas de audio digitales, consolidando conocimientos fundamentales en DSP.

Sin embargo, el proyecto presenta limitaciones derivadas de las capacidades del hardware y del enfoque académico. Las restricciones del PSoC 5LP, como su velocidad de procesamiento, memoria limitada y la resolución del ADC y DAC, afectan la calidad y complejidad de los algoritmos implementados. Los efectos desarrollados están limitados a conceptos básicos de retardos y modulaciones, sin incluir técnicas avanzadas o métodos más sofisticados usados en la industria. Además, el alcance se centra en efectos específicos de audio y no aborda otras categorías como procesamiento espectral. Por último, las pruebas auditivas se realizaron en condiciones controladas, lo que puede limitar la evaluación del sistema en contextos más exigentes o profesionales.

5. Marco Teórico

5.1. PSoC 5LP

El PSoC 5LP es un microcontrolador sumamente versátil, que combina componentes analógicos y digitales configurables con un núcleo de procesamiento ARM Cortex-M3. Gracias a esta arquitectura, es especialmente adecuado para aplicaciones que requieren procesamiento de señales avanzadas. Entre sus principales características se destacan:

- Módulos de Procesamiento de Señales Digitales (DSP): Estos módulos facilitan la implementación de filtros digitales FIR (Respuesta al Impulso Finita), lo cual es ideal para el procesamiento de señales con requisitos específicos de filtrado.
- Convertidores A/D y D/A integrados: Estos convertidores permiten capturar señales analógicas del entorno y también generar señales analógicas de salida, facilitando la interacción con diversos sensores y actuadores en aplicaciones embebidas.
- Arquitectura Flexible y Reconfigurable: La estructura del PSoC 5LP permite adaptar el diseño de sus componentes para cumplir con diferentes especificaciones de un proyecto sin necesidad de cambiar el hardware. Esto proporciona una gran ventaja en términos de adaptabilidad y personalización para distintas aplicaciones.

Además, el entorno de desarrollo PSoC Creator permite diseñar tanto el hardware configurable como el software, integrando todo en una única plataforma. Gracias a estas características, el PSoC 5LP es una excelente opción para aplicaciones en sistemas de control, procesamiento de señales, y desarrollo de prototipos en el ámbito de la electrónica.



Figura 1: PSoC 5LP

5.2. Efecto de audio

Un efecto de audio es un procesamiento o manipulación que se aplica a una señal de audio para alterar o modificar su sonido. Estos efectos pueden ser utilizados para mejorar, transformar, o simplemente dar un carácter diferente al sonido original. Los efectos de audio son fundamentales en la producción musical, la postproducción de audio en cine y televisión, y en otros ámbitos de la ingeniería de sonido.

Los efectos pueden clasificarse de muchas maneras, como por ejemplo efectos basados en retardo o efectos que modifican la señal de audio en diferentes dominios, como el tiempo, la frecuencia, la amplitud o incluso la fase.

En el trabajo nos centraremos específicamente en los efectos por retardo y por modulación.

5.3. Retardos

El procesamiento basado en retardos en señales de audio se centra en el uso de copias retardadas de la señal original, lo que permite manipular su estructura temporal y de frecuencia para crear efectos sonoros específicos. Dependiendo del tiempo de retardo y la ganancia aplicada a estas copias, se pueden generar una variedad de efectos útiles en la producción de audio.

La estructura más común para implementar un retardo es el filtro comb, que se obtiene sumando la señal original con una copia de sí misma que ha sido retrasada en el tiempo. La principal característica de este tipo de filtro es su respuesta en frecuencia, que toma la forma de un peine debido a la interferencia entre la señal original y las copias retrasadas. Esta interferencia produce una serie de picos y valles en la frecuencia, creando un efecto distintivo que es la base de muchos efectos de audio basados en retardo. Este filtro se utiliza ampliamente en la creación de efectos de audio como coro, eco, flanger, y otros, ya que permite manipular la fase y el tiempo de las señales de manera que se generen resultados sonoros complejos a partir de una simple interacción entre la señal original y su versión retardada. Los filtros comb se pueden identificar de acuerdo al tipo de señal sumada a la entrante. Si sólo depende de los valores previos en la entrada se denomina feedforward o filtro FIR, y si depende sólo de los valores previos de la salida se llama

feedback o filtro IIR.

5.4. Filtrado digital

5.4.1. Filtros FIR

Un filtro FIR (Finite Impulse Response) es un tipo de filtro digital el cual cuenta con una respuesta al impulso finita. Este filtro tiene todos los polos en el origen, lo que lo vuelve un filtro estable y hace que sea de interés para aplicaciones en audio.

5.4.2. Filtros IIR

Un filtro IIR (Infinite Impulse Response) es un filtro digital el cual cuenta con una respuesta al impulso infinita. A diferencia de los filtros Fir, este filtro cuenta con polos y ceros que determinan su estabilidad.

5.4.3. Filtros comb FIR

Un filtro comb FIR es un caso particular de un filtro FIR. El diagrama de bloques de un filtro comb FIR (o también denominado filtro FeedForward) es el siguiente:

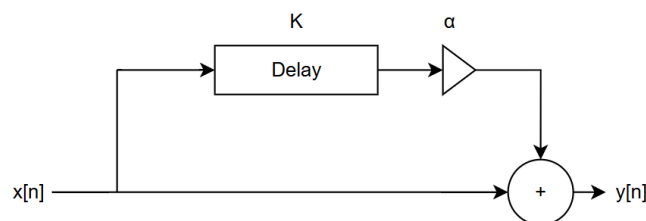


Figura 2: Diagrama de bloques del filtro FIR. ©2024, Autoría propia.

Dicho diagrama puede ser descrito por la ecuación en diferencias

$$y[n] = x[n] + \alpha * x[n - K]$$

y su respuesta en frecuencia

$$H(e^{jw}) = 1 + \alpha * e^{-jwK}$$

donde

- α : Es una constante que representa la ganancia en el filtro.
- K : Es el coeficiente que ajusta el retraso de la señal.

A partir de la respuesta en frecuencia se puede obtener el siguiente gráfico en Matlab variando el valor de α , en el cual se puede apreciar la forma de peine (comb):

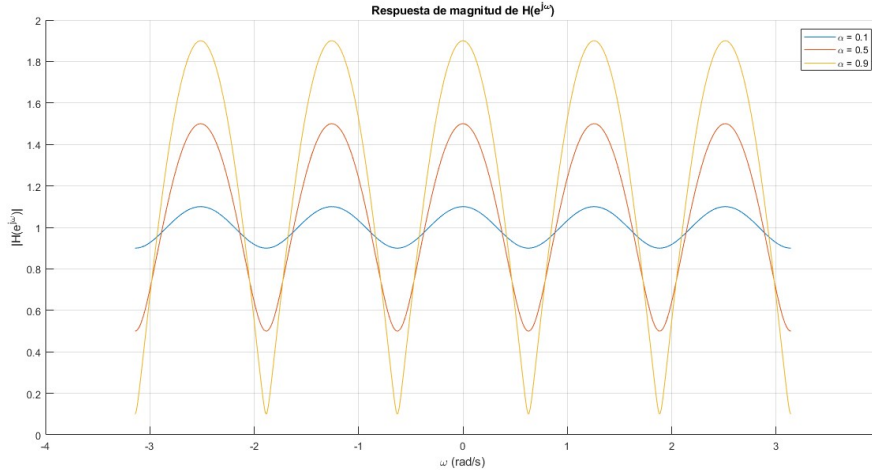


Figura 3: Respuesta en magnitud del filtro comb FIR.

5.4.4. Filtros comb IIR

Un filtro comb IIR es un caso particular de un filtro IIR. El diagrama de bloques de un filtro comb IIR (o también denominado filtro FeedBackward) es el siguiente:

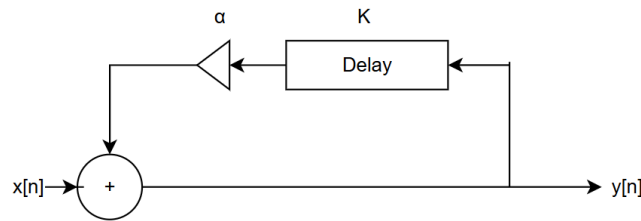


Figura 4: Diagrama de bloques del filtro IIR. ©2024, Autoría propia.

Dicho diagrama puede ser descrito por la ecuación en diferencias

$$y[n] = x[n] + \alpha * y[n - K]$$

y su respuesta en frecuencia

$$H(e^{jw}) = \frac{1}{1 - \alpha * e^{-jwK}}$$

donde

- α : Es una constante que representa la ganancia en el filtro.
- K : Es el coeficiente que ajusta el retraso de la señal.

A partir de la respuesta en frecuencia se puede obtener el siguiente gráfico en Matlab variando el valor de α , en el cual se puede apreciar la forma de peine (comb):

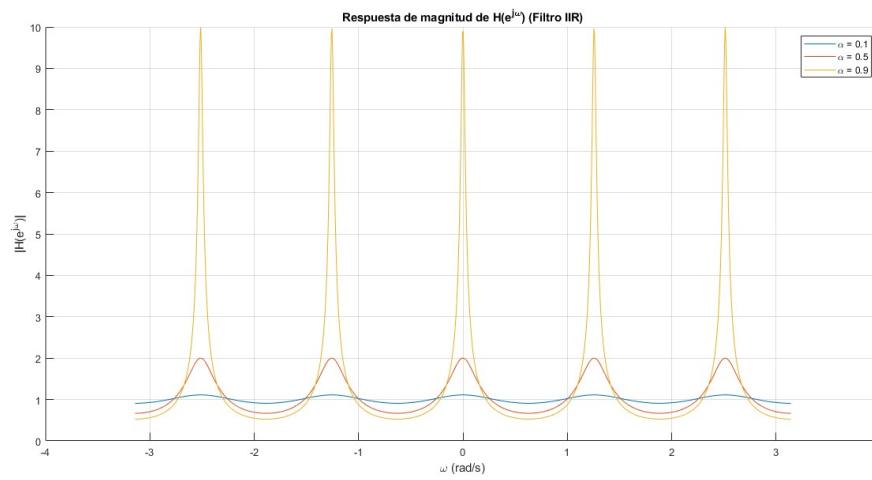


Figura 5: Respuesta en magnitud del filtro comb IIR.

5.5. Modulación

La modulación es un proceso fundamental en el procesamiento de señales, que consiste en variar ciertos parámetros de una señal portadora (como la amplitud, frecuencia o fase) en función de otra señal, conocida como la señal moduladora. Este concepto es ampliamente utilizado en efectos de audio para introducir variaciones controladas y dinámicas en la señal original. Dependiendo de los parámetros modulados, la modulación puede alterar el tono, la amplitud o el tiempo de la señal, generando una amplia gama de efectos sonoros. Estos efectos modulados son esenciales para la creación de texturas sonoras complejas, proporcionando un elemento de movimiento y variabilidad que enriquece la experiencia auditiva.

5.6. Efectos a ser implementados

5.6.1. Echo

El eco es uno de los efectos más comunes basados en retardo. Consiste en reproducir la señal de audio con un retardo específico, creando una repetición del sonido original después de un breve período de tiempo. Este efecto es utilizado para simular el sonido reflejado de una superficie (como en un cañón o una cueva).

Para el efecto, la ecuación de diferencias del filtro comb FIR a ser utilizado se expresa como

$$y[n] = x[n] + \alpha_1 * x[n - K_1] + \alpha_2 * x[n - K_2]$$

5.6.2. Flanger

El flanger es similar al chorus, pero con un retardo más corto y modulable. Produce un sonido característico de ondas que cambian en el tiempo. La señal original se combina con una copia de sí misma, pero la amplitud de la misma se modula de forma cíclica.

Para el efecto, la ecuación de diferencias del filtro comb FIR a ser utilizado se expresa como

$$y[n] = x[n] + \alpha * m * x[n] + x[n - K]$$

donde el término modulado m está dado por

$$m = \cos(2\pi * f \frac{n}{K})$$

5.6.3. Chorus

El efecto de chorus crea la ilusión de que varias voces o instrumentos están tocando o cantando al mismo tiempo, pero en realidad se utiliza una única fuente de sonido modificada. Esto se logra combinando la señal original con copias ligeramente retrasadas y variando el retardo.

Para el efecto, la ecuación de diferencias del filtro comb IIR a ser utilizado se expresa como

$$y[n] = x[n] + \alpha * y[n - K]$$

5.6.4. Vibrato

El vibrato es un efecto de modulación de frecuencia en el que el tono de la señal varía periódicamente. Aunque en esencia no es un retardo, a menudo se genera mediante un retardo modulado en la frecuencia, lo que lo hace un tipo de filtro de audio por retardado.

Para el efecto, la ecuación de diferencias del filtro comb FIR a ser utilizado se expresa como

$$y[n] = x[n - m(n)]$$

donde la modulación del retardo está dado por

$$m(n) = \frac{D}{2} * (1 + \sin(2\pi * f_v \frac{n}{f_s}))$$

donde

- D : Máximo retardo en muestras.
- f_v : Frecuencia del vibrato.
- f_s : Frecuencia de muestreo.

5.7. Aliasing

El aliasing, frecuentemente mencionado como distorsión por aliasing, se presenta al procesar señales de forma digital cuando una señal continua es convertida en digital a una frecuencia muy baja. Al omitir ciertas características de la señal original, en particular aquellas con frecuencias más altas, el proceso de muestreo a baja frecuencia puede resultar en la representación errónea de algunas frecuencias en la señal convertida a formato digital.

La frecuencia de Nyquist se sitúa en la mitad de la frecuencia de muestreo y establece el límite mínimo para capturar una señal de forma precisa sin distorsiones. En otras palabras, para representar adecuadamente una señal y evitar el aliasing, se recomienda

que la frecuencia de muestreo sea al menos el doble de la frecuencia máxima en la señal original. De no cumplirse esta condición, las frecuencias altas de la señal original se despliegan hacia el rango de frecuencias más bajas y se muestran de manera errónea en la señal digitalizada. De esta manera, en vez de apreciar las verdaderas frecuencias de la señal original, percibimos frecuencias ficticias que provocan una representación distorsionada y poco fidedigna de la señal.

Este fenómeno se puede comprender fácilmente en el contexto de las aplicaciones de audio. Si una señal de audio tiene frecuencias altas pero se muestrea a una frecuencia baja, es posible que esas frecuencias se interpreten como tonos más bajos o creen ruidos no deseados al convertirse en formato digital.

Existe un conjunto de estrategias que pueden ser utilizadas para prevenir el aliasing. Una de las opciones es la aplicación de filtros anti-aliasing. Estos filtros se encargan de suprimir las frecuencias más elevadas de la señal, específicamente las que superan la frecuencia de Nyquist, justo antes del proceso de muestreo. De este modo, se asegura que únicamente las frecuencias autorizadas sean consideradas en el proceso de muestreo, evitando así el aliasing. Otra táctica consiste en garantizar que la frecuencia de muestreo sea al menos el doble de la frecuencia máxima de la señal original, asegurando así cumplir con el principio de Nyquist. Esto posibilita la captura y reconstrucción de la señal digital sin alteraciones, logrando una representación precisa de la señal analógica en el ámbito digital.

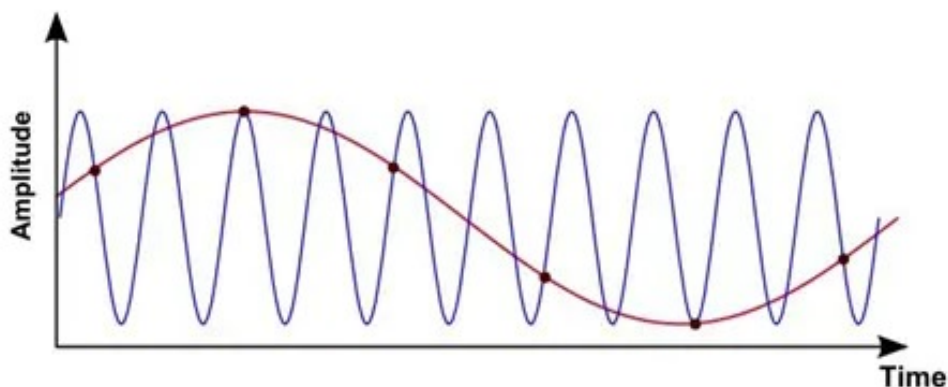


Figura 6: Efecto del aliasing.

5.8. Esquema general de la solución

Para poder aplicar los efectos deseados sobre la señal de audio, se estará utilizando el PSoC 5LP, el mismo lo que hará es digitalizar la señal de entrada, previamente filtrada para evitar el aliasing, y luego aplicará los filtros correspondientes para obtener los efectos de audio deseados, posteriormente se pasará nuevamente a una señal analógica, la cual pasará nuevamente por un filtro rector, siendo la salida de este, la salida del sistema. A continuación se presenta el diagrama de bloques del proyecto.

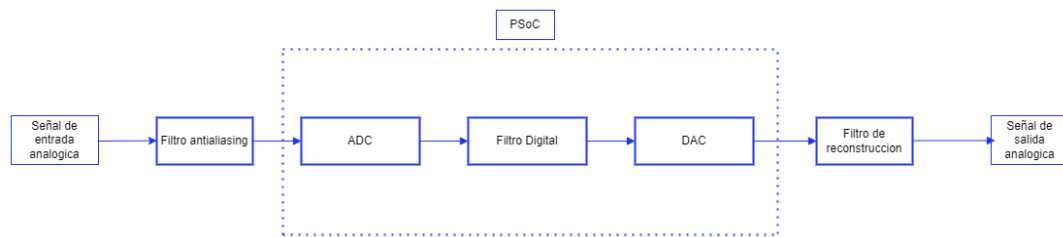


Figura 7: Diagrama de bloques simplificado de la configuración del PSoC. ©2024, Autoría propia.

5.8.1. Bloques ADC

El PSoC 5LP cuenta con un bloque ADC (Convertidor Analógico a Digital) integrado. Este ADC es fundamental para la conversión de señales analógicas en datos digitales, lo que permite al microcontrolador procesar señales analógicas como audio, temperatura, entre otras.

El ADC integrado en el PSoC 5LP usa un convertidor SAR (Successive Approximation Register), que es uno de los métodos más comunes para la conversión de señales analógicas en digitales. El proceso consiste en aproximar gradualmente el valor analógico a su valor digital. En cada paso, el ADC compara el valor de la señal analógica con una referencia interna, refinando su estimación hasta que alcanza una precisión suficientemente alta.

La resolución del ADC del PSoC 5LP es de hasta 12 bits, lo que significa que el valor digital resultante de la conversión puede variar entre 0 y 4095 ($2^{12} - 1$). Esto le permite detectar cambios pequeños en la señal analógica, lo que es útil para aplicaciones que requieren alta precisión, como las mediciones de señales débiles.

5.8.2. Bloques VDAC

El bloque VDAC (Convertidor Digital a Analógico) del PSoC 5LP es un componente que convierte señales digitales en señales analógicas. El VDAC toma un valor digital de entrada y lo convierte en un voltaje proporcional a ese valor.

El funcionamiento básico de un VDAC implica la conversión de un número binario a una señal de voltaje continua. El VDAC opera utilizando una referencia de voltaje interna o externa para generar el voltaje de salida correspondiente.

La resolución estándar del VDAC en el PSoC 5LP es de 8 bits, lo que significa que la señal de salida puede tener 256 niveles distintos. Sin embargo, en ciertas configuraciones, también es posible utilizar resoluciones mayores, como 12 bits, mediante el uso de técnicas adicionales, pero la resolución estándar es 8 bits. Esto permite un control adecuado de señales analógicas en muchas aplicaciones, incluyendo el procesamiento de audio y control de voltajes.

5.8.3. Filtro antialiasing

Como este proyecto trabaja con audios, es de suma importancia la utilización de un filtro antialiasing, debido a que el rango de frecuencias que el ser humano puede percibir es de 20 Hz a 20 kHz, y debemos de ajustar a eso en la señal de audio que recibimos en la entrada, para poder así evitar el efecto del aliasing.

Para lograr hacer este filtro estaremos utilizando un filtro pasa bajas Butterworth de segundo orden implementado mediante Sallen Key con ganancia unitaria.

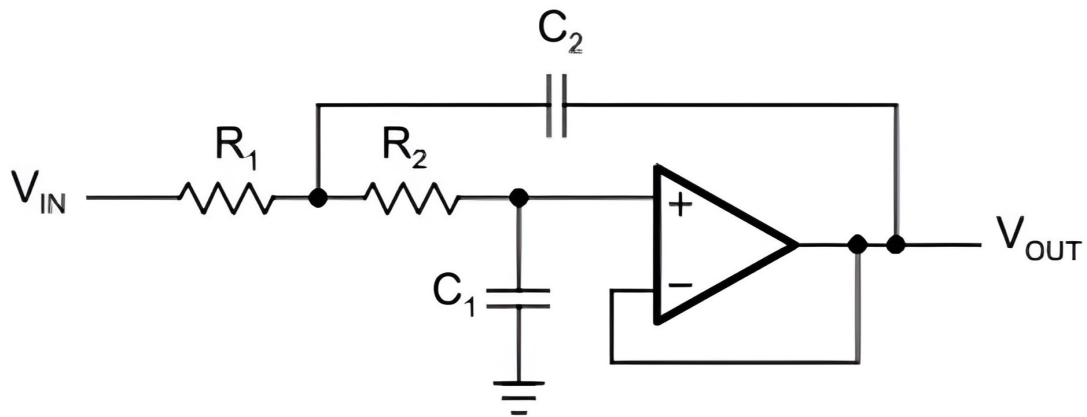


Figura 8: Butterworth mediante Sallen-Key para filtro pasa bajas de segundo orden.

La ecuacion que define la frecuencia de corte de este filtro es el siguiente:

$$f_c = \frac{1}{2 * \pi * \sqrt{R1 * R2 * C1 * C2}}$$

Por criterio de diseño se hara que $R1=R2=R$ y que $2C1=2C=C2$, de tal forma la ecuacion queda:

$$f_c = \frac{1}{2 * \pi * R * C * \sqrt{2}}$$

La frecuencia de corte buscada es de aproximadamente de 20kHz, y se tomara el valor de capacitor igual a 100nF, entonces la ecuacion queda de la siguiente manera:

$$20 * 10^3 = \frac{1}{2 * \pi * R * 100 * 10^{-9} * \sqrt{2}}$$

Al resolver la ecuacion se obtiene que

$$R \approx 56\Omega$$

El diseño de este filtros se hizo utilizando el software OrCAD Capture Lite, dicho diseño se puede ver en acontinuacion.

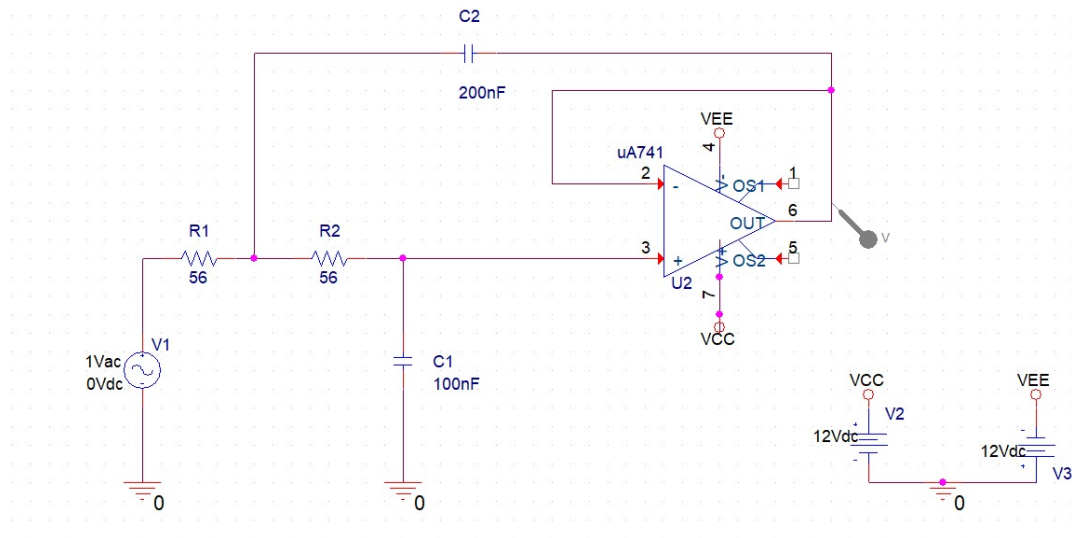


Figura 9: Diseño del circuito en OrCAD.

A continuación, se obtiene la respuesta en frecuencia del filtro diseñado para así comprobar su correcto funcionamiento, para ello se estará observando su frecuencia de corte.

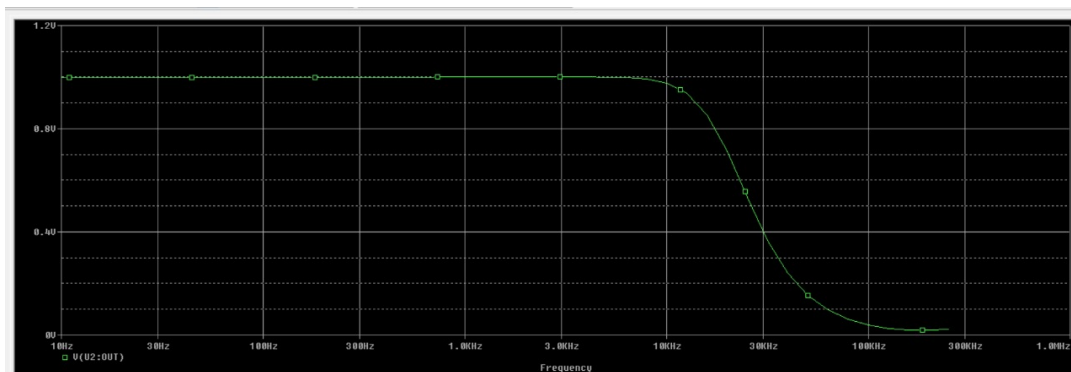


Figura 10: Respuesta en frecuencia del filtro diseñado

Como la frecuencia de corte es el punto donde la magnitud de la respuesta del filtro disminuye aproximadamente -3dB, se concluye que el filtro diseñado funciona de manera correcta y es aceptable para su uso.

Es relevante destacar que en este diseño se empleó el amplificador operacional uA741, debido a su capacidad para simular la respuesta en frecuencia en OrCAD. Sin embargo, este componente presenta múltiples desventajas, lo que justifica su exclusión en la implementación final del proyecto.

5.8.4. Amplificador Operacional LM324

El LM324 es un circuito integrado que contiene cuatro amplificadores operacionales independientes diseñados para aplicaciones de propósito general. Los amplificadores incluidos en el LM324 presentan una alta impedancia de entrada, bajo consumo de corriente (1 mA por amplificador) y la capacidad de operar cerca del nivel de tierra en configuraciones de alimentación simple, lo que los hace ideales para aplicaciones como amplificación de señales, filtrado analógico, generación de señales y sistemas de control. Aunque su ancho de banda y velocidad de respuesta son limitados (1 MHz y 0.5 V/ μ s, respectivamente), es una solución confiable y económica para proyectos de baja frecuencia, como audio y sensores analógicos.

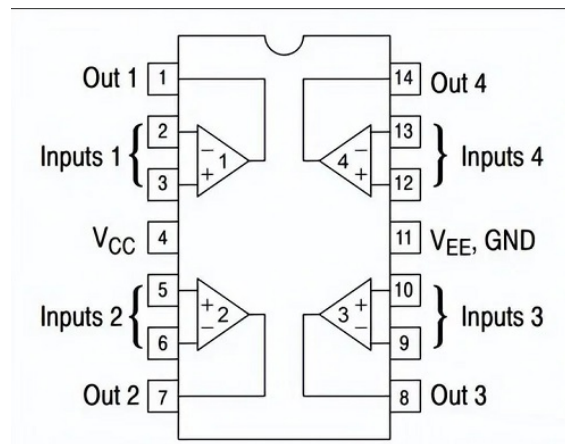


Figura 11: Distribución de pines del LM324.

El LM324 es una opción práctica y eficiente para este proyecto, ya que combina un buen desempeño en audio con facilidad de uso, robustez y bajo costo.

5.9. Filtro de reconstrucción

Al momento de convertir una señal digital a analógica mediante un DAC, se genera una señal analógica que contiene réplicas de la señal base alrededor de la frecuencia de muestreo y sus múltiplos.

Por lo tanto, para garantizar que la salida analógica sea limpia y no contenga ruidos no deseados, se colocó a la salida del VDAC un filtro de reconstrucción que atenúa las frecuencias superiores a las frecuencias audibles.

Para el efecto se utilizó el mismo diseño utilizado para el filtro antialiasing, debido a que lo que buscamos es también un filtro pasa bajos con la misma frecuencia de corte.

6. Justificación y diseño de filtros seleccionados

6.1. Diseño a base de bloques en el PSoC Creator

A continuación se muestra la implementación del proyecto en el PSoC Creator. Se empleó el siguiente esquema de manera general para los cuatro efectos, como se muestra más abajo. Lo único que varía entre cada efecto es el algoritmo de procesamiento de la señal de entrada.

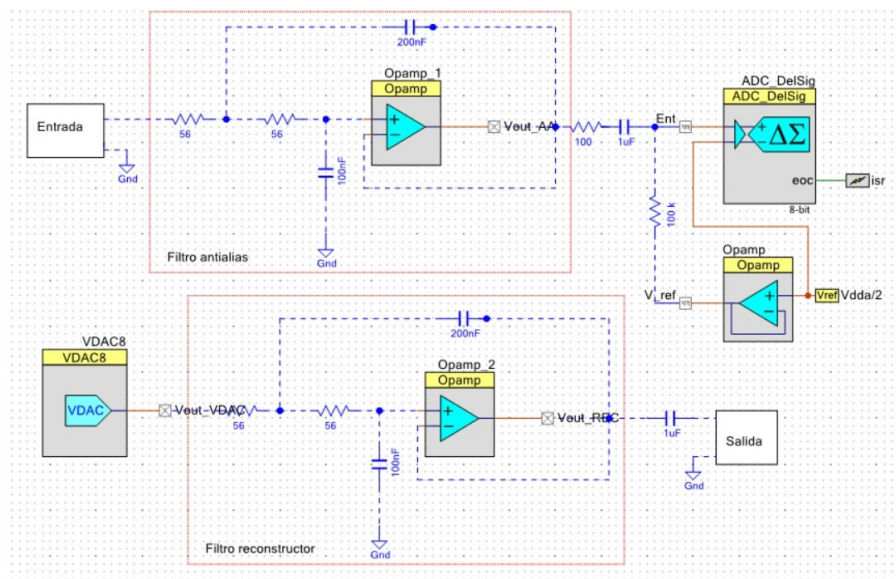


Figura 12: Top Design de la implementación en el PSoC Creator

- **Opamp:** Es un amplificador operacional interno que actúa como buffer y monta la señal de entrada sobre un voltaje continuo de 2.5 V, permitiendo la correcta digitalización por el ADC DelSig. Utiliza una resistencia de 100 k Ω y se protege con una resistencia de 100 Ω y un capacitor de 1 μ F.
- **ADC DelSig:** El convertidor analógico a digital de tipo Delta-Sigma tiene una resolución de 8 bits, para el presente proyecto fue configurada una frecuencia de muestreo de 48kHz, de manera de cumplir con la frecuencia de Nyquist.
- **VDAC:** Los convertidores digital-analógico reciben un dato de 8 bits en su registro de entrada y lo convierten a un valor de voltaje, recreando la señal analógica en un pin del PSoC.

Tanto el filtro antialiasing como el filtro de reconstrucción se implementan externamente en el sistema. Los algoritmos implementados para cada efecto están fundamentados en las ecuaciones en diferencias descritas en los apartados previos.

6.2. Justificación de filtros seleccionados

Inicialmente, para cada efecto se definieron las ecuaciones en diferencias correspondientes. Sin embargo, al implementarlas en la práctica, surgieron varios inconvenientes. Algunos de los problemas observados fueron: el efecto era casi imperceptible al oído, ciertos efectos diferentes resultaban muy similares entre sí, la señal de audio de salida presentaba un alto nivel de ruido, la memoria RAM del PSoC no era suficiente, entre otros. Por estas razones, fue necesario realizar mejoras en varios efectos.

Por ejemplo, la ecuación en diferencias inicial para el efecto de eco era:

$$y[n] = \alpha \cdot y[n - K] + x[n],$$

pero se mejoró añadiendo un término adicional, como se describió previamente en la sección de efectos implementados.

Otro ajuste significativo se realizó en el efecto flanger, cuya ecuación en diferencias original era:

$$y[n] = x[n] + \alpha \cdot x[n - m(n)],$$

donde $m(n)$ representaba un retardo modulado. Al final, esta ecuación fue modificada considerablemente para obtener un mejor resultado en la práctica.

6.3. Implementación en MATLAB

A continuación se describen los códigos en MATLAB de cada efecto implementado en este proyecto.

6.3.1. Echo

```
1 clear all
2 close all
3 clc
```

```

4 %% Lectura del audio
5 [x, fs] = audioread("Recorte.wav"); % Cargar el archivo de audio
6 t = length(x) / fs; % Duraci n del audio en segundos
7 %% Parmetros del eco
8 delay1_ms = 75; % Primer retardo en milisegundos (3600 muestras
    a 48 kHz)
9 delay2_ms = 125; % Segundo retardo en milisegundos (6000
    muestras a 48 kHz)
10 alpha1 = 0.7; % Atenuaci n del primer eco
11 alpha2 = 0.4; % Atenuaci n del segundo eco
12 offset = 2.04; % Offset constante
13 % Convertir retardos a muestras
14 delay1 = round((delay1_ms / 1000) * fs);
15 delay2 = round((delay2_ms / 1000) * fs);
16 % Inicializar la se al de salida
17 N = length(x);
18 y = zeros(size(x)); % Senhal de salida con efecto eco
19 %% Aplicar el efecto eco
20 y(1:delay2, :) = x(1:delay2, :); % Copiar la parte inicial sin
    eco
21 % Procesar cada muestra despues del retardo mayor
22 for n = delay2+1:N
23     y(n, :) = x(n, :) + ...
24         alpha1 * x(n - delay1, :) + ...
25         alpha2 * x(n - delay2, :) + ...
26         offset; % Ecuacion del eco
27 end
28 % Normalizar para evitar saturacion
29 y = y / max(abs(y(:)));
30 %% Concatenar el audio original y el audio con efecto eco
31 audio_final = [x; y]; % Concatenar el audio original seguido del
    audio con eco
32 sound(audio_final, fs); % Reproducir el audio concatenado

```

```
33 %% Grafica del audio original y el audio con eco
34 figure
35 subplot(2, 1, 1)
36 plot(x(:, 1))
37 title('Original')
38 subplot(2, 1, 2)
39 plot(y(:, 1))
40 title('Eco Effect')
```

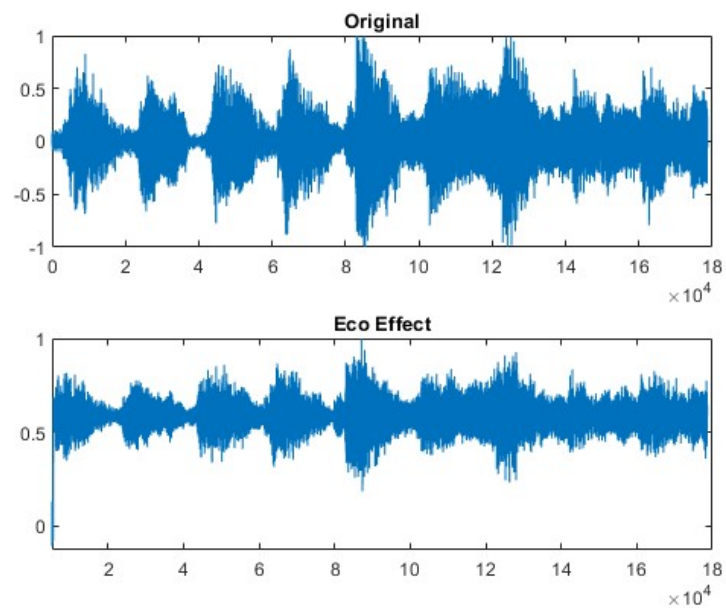


Figura 13: Comparación del audio original con el efecto Echo.

6.3.2. Flanger

```
1 clear all
2 close all
3 clc
4 %% Lectura del audio
5 [x, fs] = audioread("Recorte.wav"); % Cargar el archivo de audio
6 t = length(x) / fs; % Duracion del audio en segundos
7 %% Parametros del flanger
8 D = 960; % Retardo maximo en muestras (~20 ms a 48 kHz)
9 f_flanger = 0.25; % Frecuencia de modulacion en Hz
10 alpha = 0.7; % Atenuacion del retardo
11 N = length(x);
12 y = zeros(size(x)); % Inicializar la se al de salida
13 % Aplicar el efecto flanger
14 for n = D+1:N
15     d = round(D / 2 * (1 + sin(2 * pi * f_flanger * n / fs))); %
        Retardo modulado
16     y(n, :) = x(n, :) + alpha * x(n - d, :); % Ecuacion del
        flanger
17 end
18 %% Concatenar el audio original y el audio con flanger
19 audio_final = [x; y]; % Concatenar el audio original seguido del
        audio con flanger
20 sound(audio_final, fs); % Reproducir el audio concatenado
21 %% Grafica del audio original y el audio con flanger
22 figure
23 subplot(2, 1, 1)
24 plot(x(:, 1))
25 title('Original')
26 subplot(2, 1, 2)
27 plot(y(:, 1))
28 title('Flanger Effect')
```

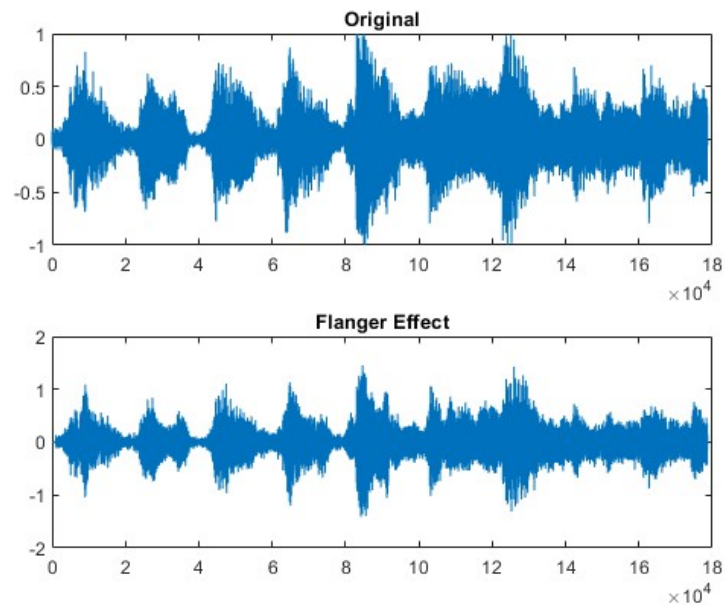


Figura 14: Comparación del audio original con el efecto Flanger.

6.3.3. Chorus

```

1 clear all
2 close all
3 clc
4 %% Lectura del audio
5 [x, fs] = audioread("Recorte.wav"); % Cargar el archivo de audio
6 t = length(x) / fs; % Duracion del audio en segundos
7 %% Parametros del chorus
8 D = 850; % Retardo base en muestras (~20 ms a 48 kHz)
9 alpha_chorus = 0.8; % Atenuacion del retardo
10 N = length(x);
11 y = zeros(size(x)); % Inicializar la se al de salida
12 % Aplicar el efecto chorus
13 for n = D+1:N
14     d = round(D / 2 * (1 + sin(2 * pi * 0.3 * n / fs))); %
        Retardo modulado
15     y(n, :) = x(n, :) + alpha_chorus * x(n - d, :); % Ecuacion
        del chorus

```

```
16 end
17 %% Concatenar el audio original y el audio con chorus
18 audio_final = [x; y]; % Concatenar el audio original seguido del
    audio con chorus
19 sound(audio_final, fs); % Reproducir el audio concatenado
20 %% Grafica del audio original y el audio con chorus
21 figure
22 subplot(2, 1, 1)
23 plot(x(:, 1))
24 title('Original')
25 subplot(2, 1, 2)
26 plot(y(:, 1))
27 title('Chorus Effect')
```

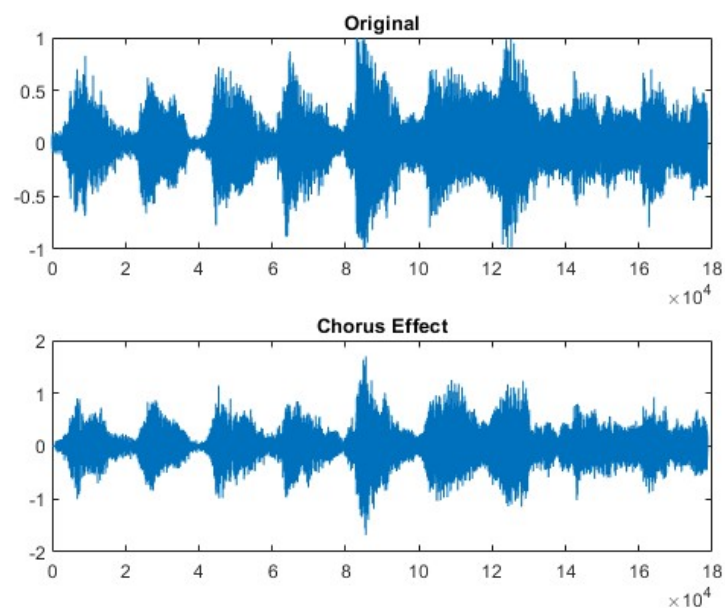


Figura 15: Comparación del audio original con el efecto Chorus

6.3.4. Vibrato

```
1 clear all
2 close all
3 clc
4 %% Lectura del audio
5 [x, fs] = audioread("Recorte.wav"); % Cargar el archivo de audio
6 t = length(x) / fs; % Duracion del audio en segundos
7 %% Parametros del vibrato
8 D = 960; % Retardo maximo en muestras (~20 ms a 48 kHz)
9 f_vibrato = 5; % Frecuencia del vibrato en Hz
10 N = length(x);
11 y = zeros(size(x)); % Inicializar la senhal de salida
12 % Aplicar el efecto vibrato
13 for n = D+1:N
14     d = round(D / 2 * (1 + sin(2 * pi * f_vibrato * n / fs))); %
        Retardo modulado
15     y(n, :) = x(n - d, :); % Ecuacion del vibrato
16 end
17 %% Concatenar el audio original y el audio con vibrato
18 audio_final = [x; y]; % Concatenar el audio original seguido del
        audio con vibrato
19 sound(audio_final, fs); % Reproducir el audio concatenado
20 %% Grafica del audio original y el audio con vibrato
21 figure
22 subplot(2, 1, 1)
23 plot(x(:, 1))
24 title('Original')
25 subplot(2, 1, 2)
26 plot(y(:, 1))
27 title('Vibrato Effect')
```

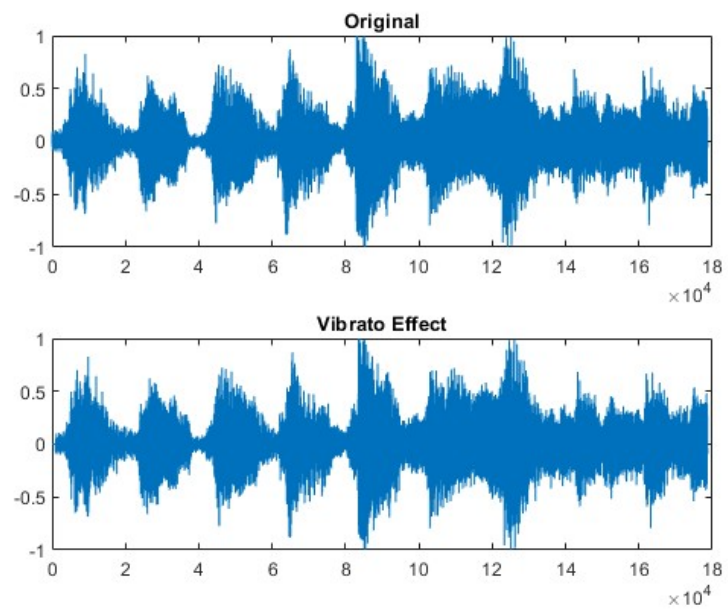


Figura 16: Comparación del audio original con el efecto Vibrato.

6.4. Implementación en PSoC

A continuación se describen los códigos en PSoC Creator de cada efecto implementado en este proyecto

6.4.1. Echo

```
1 #include "project.h"
2 #define RETARDO_1 3600
3 #define RETARDO_2 6000
4 float y;
5 float entrada = 0.0;
6 float x_delayed_1[RETARDO_1] = {0};
7 float x_delayed_2[RETARDO_2] = {0};
8 int retardo_index_1 = 0;
9 int retardo_index_2 = 0;
10 float OFFSET = 2.04;
11 volatile uint8_t flag = 0;
12 CY_ISR_PROTO(Adquisicion);
13 int main(void) {
14     ADC_DelSig_IRQ_Start();
15     isr_StartEx(Adquisicion);
16     ADC_DelSig_Start();
17     ADC_DelSig_StartConvert();
18     Opamp_Start();
19     VDAC8_Start();
20     isr_Enable();
21     CyDelay(50);
22     CyGlobalIntEnable;
23
24     for (;;) {
25         if (flag == 1) {
26             entrada = ADC_DelSig_CountsTo_Volts(
                ADC_DelSig_GetResult8());
```

```

27         y = entrada + 0.7 * x_delayed_1[retardo_index_1] +
           0.4 * x_delayed_2[retardo_index_2] + OFFSET;
28         // Actualizar valores
29         x_delayed_1[retardo_index_1] = entrada;
30         x_delayed_2[retardo_index_2] = entrada;
31         retardo_index_1++;
32         retardo_index_2++;
33         if (retardo_index_1 >= RETARDO_1) {
34             retardo_index_1 = 0;
35         }
36         if (retardo_index_2 >= RETARDO_2) {
37             retardo_index_2 = 0;
38         }
39         // Limitar la salida al rango permitido
40         if (y >= 4.08) {
41             y = 4.08;
42         }
43         if (y <= 0.0) {
44             y = 0.0;
45         }
46         // Escalar y enviar la salida al ADC
47         uint8_t val = (uint8_t)(y * 255.0 / 4.08);
48         VDAC8_SetValue(val);
49         flag = 0;
50     }
51 }
52 }
53 // Interrupcion del ADC
54 CY_ISR_PROTO(Adquisicion) {
55     flag = 1;
56     isr_ClearPending();
57 }

```

6.4.2. Flanger

```
1 #include "project.h"
2 #include <math.h>
3
4 #define RETRASO (480) // Delay length for the second term (10ms
   * 48ksps)
5 float y;
6 float entrada = 0.0; // Single value for x[n]
7 float x_delayed[RETRASO] = {0}; // Delay line for the second
   term
8 int delay_i = 0;
9 float OFFSET = 2.0; // Adjust the offset as needed
10 volatile uint8_t flag = 0; // Define flag as a global variable
11
12 CY_ISR_PROTO(Adquisicion);
13
14 int main(void) {
15     ADC_DelSig_IRQ_Start();
16     isr_StartEx(Adquisicion);
17     ADC_DelSig_Start();
18     ADC_DelSig_StartConvert();
19     Opamp_Start();
20     VDAC8_Start();
21     isr_Enable();
22     CyDelay(50);
23     CyGlobalIntEnable;
24
25     for (;;) {
26         if (flag == 1) {
27             entrada = ADC_DelSig_CountsTo_Volts(
28                 ADC_DelSig_GetResult8());
29             float cos_term = 0.7*cos(2 * M_PI * 0.45 * delay_i /
```

```

        480.0);
29      y = entrada + 0.9*entrada * cos_term + x_delayed[
        delay_i] + OFFSET;
30      // Actualizar valores
31      x_delayed[delay_i] = entrada;
32      delay_i++;
33      if (delay_i >= RETRASO) {
34          delay_i = 0;
35      }
36      // Limitar la salida al rango permitido
37      if (y >= 4.08) {
38          y = 4.08;
39      }
40      if (y <= 0.0) {
41          y = 0.0;
42      }
43      // Escalar y enviar la salida al ADC
44      uint8_t val = (uint8_t)(y * 255.0 / 4.08);
45      VDAC8_SetValue(val);
46      flag = 0;
47  }
48  }
49  }
50  // Interrupcion del ADC
51  CY_ISR_PROTO(Adquisicion) {
52      flag = 1;
53      isr_ClearPending();
54  }

```

6.4.3. Chorus

```
1 #include "project.h"
2 #include <math.h>
3 #include <stdbool.h>
4
5 #define RETARDO 850
6 #define F_MUESTREO 44100.0 // Tasa de muestreo en Hz
7 #define OFFSET 2.04
8 float y;
9 float alpha = 0.8; // Ganancia del coro
10 float y_delayed[RETARDO] = {0};
11 int delay_i = 0;
12 volatile uint8_t flag = 0;
13
14 CY_ISR_PROTO(Adquisicion);
15
16 int main(void) {
17     ADC_DelSig_IRQ_Start();
18     isr_StartEx(Adquisicion);
19     ADC_DelSig_Start();
20     ADC_DelSig_StartConvert();
21     Opamp_ADC_Start();
22     VDAC8_Start();
23     isr_Enable();
24     CyDelay(50);
25     CyGlobalIntEnable;
26
27     float t = 0;
28     float delta_t = 1.0 / F_MUESTREO; // Incremento de tiempo
        por muestra
29
30     for (;;) {
```

```
31     float valor = ADC_DeISig_CountsTo_Volts(  
        ADC_DeISig_GetResult8());  
32     y = valor + 1.5*alpha * y_delayed[delay_i] + OFFSET;  
33  
34  
35     // Actualizar valores  
36     y_delayed[delay_i] = valor;  
37     delay_i = (delay_i + 1) % RETARDO;  
38     // Limitar la salida al rango permitido  
39     if (y > 4.08) y = 4.08;  
40     if (y < 0.0) y = 0.0;  
41     // Escalar y enviar la salida al ADC  
42     uint8_t val = (uint8_t)(y * 255.0 / 4.08);  
43     VDACC8_SetValue(val);  
44     flag = 0;  
45     }  
46 }  
47  
48  
49 // Interrupcion del ADC  
50 CY_ISR(Adquisicion) {  
51     flag = 1;  
52     isr_ClearPending();  
53 }
```

6.4.4. Vibrato

```
1 #include "project.h"
2 #include <math.h>
3
4 #define DELAY (960) // Maximo retardo en muestras
5 #define FREC_V 5.0 // Frecuencia del vibrato en Hz
6 #define FS 48000.0 // Frecuencia de muestreo en Hz
7 float buffer[DELAY] = {0};
8 int delay_i = 0;
9 float OFFSET = 2.0;
10 volatile uint8_t flag = 0;
11
12 CY_ISR_PROTO(Adquisicion);
13
14 int main(void) {
15     ADC_DelSig_IRQ_Start();
16     isr_StartEx(Adquisicion);
17     ADC_DelSig_Start();
18     ADC_DelSig_StartConvert();
19     Opamp_Start();
20     VDAC8_Start();
21     isr_Enable();
22     CyDelay(50);
23     CyGlobalIntEnable;
24
25     float lectura, muestra_r, vibrato_delay;
26     int actual_delay;
27     for (;;) {
28         if (flag == 1) {
29             // Leer la muestra actual desde el ADC
30             lectura = ADC_DelSig_CountsTo_Volts(
                ADC_DelSig_GetResult8());
```

```

31         // Calcular el retardo
32         vibrato_delay = (DELAY / 2.0) * (1.0 + sinf(2 * M_PI
           * FREC_V * delay_i / FS));
33         actual_delay = (int)vibrato_delay;
34         // Obtener la muestra retardada
35         int delayed_index = (delay_i - actual_delay + DELAY)
           % DELAY;
36         muestra_r = buffer[delayed_index];
37         float salida = muestra_r + OFFSET;
38         // Guardar la muestra actual en el buffer
39         buffer[delay_i] = lectura;
40         // Incrementar el indice de retardo
41         delay_i = (delay_i + 1) % DELAY;
42         // Limitar la salida al rango permitido
43         if (salida >= 4.08) {
44             salida = 4.08;
45         }
46         if (salida <= 0.0) {
47             salida = 0.0;
48         }
49         // Escalar y enviar la salida al DAC
50         uint8_t val = (uint8_t)(salida * 255.0 / 4.08);
51         VDAC8_SetValue(val);
52         flag = 0;
53     }
54 }
55 }
56 // Interrupcion del ADC
57 CY_ISR(Adquisicion) {
58     flag = 1;
59     isr_ClearPending();
60 }

```

7. Montaje y resultados

7.1. Montaje final

Utilizando el amplificador operacional LM324 se realizaron las conexiones para el montaje final de los filtros antialias y de reconstrucción, los cuales fueron conectados al Psoc siguiendo la lógica del diagrama de bloques visto anteriormente.

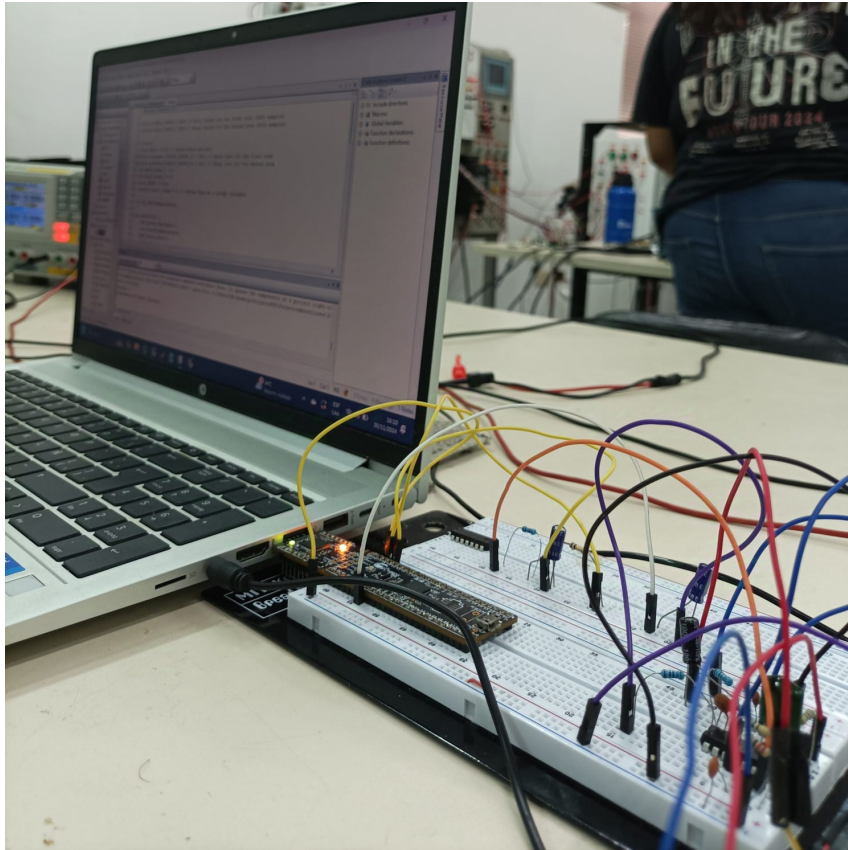


Figura 17: Montaje completo final.

7.2. Resultados obtenidos

Se realizó la verificación del funcionamiento de los filtros antialias y de reconstrucción mediante el uso de un generador de señales para la señal de entrada y un osciloscopio para observar la señal de salida.

A continuacion se muestran las salidas obtenidas del filtro antialias.

- Salida del filtro a 600Hz

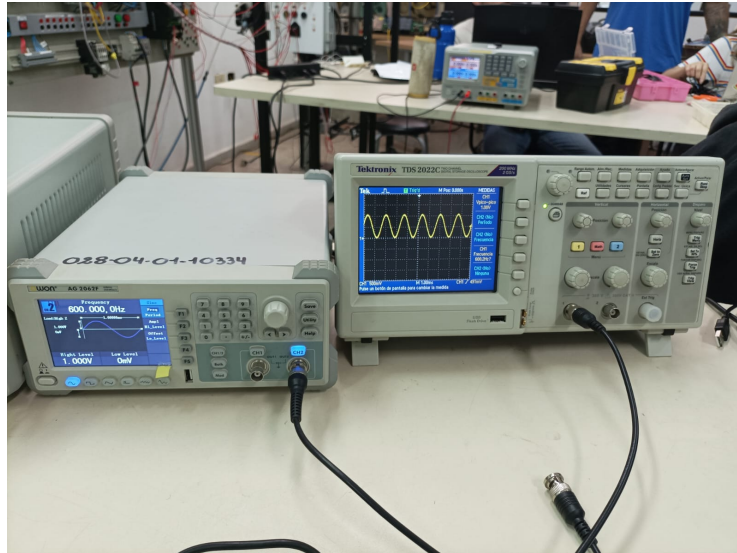


Figura 18: Prueba con una señal de 600Hz.

- Salida del filtro a 5kHz

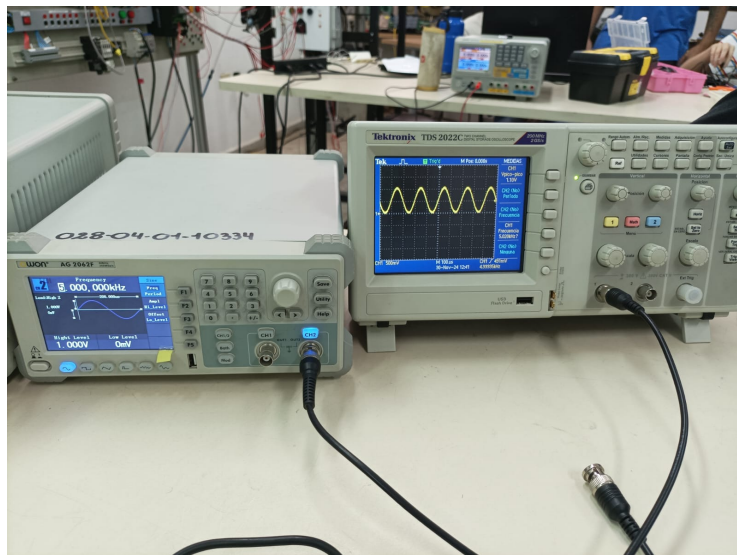


Figura 19: Prueba con una señal de 5kHz.

- Salida del filtro a 20kHz

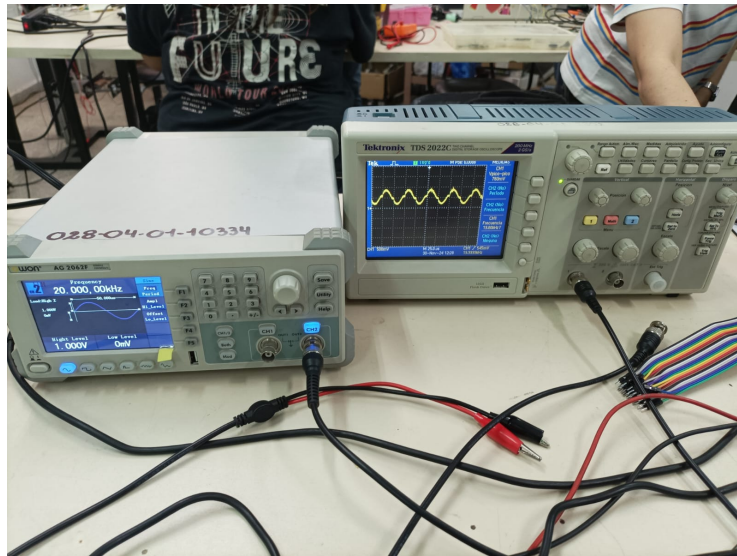


Figura 20: Prueba con una señal de 20kHz.

- Salida del filtro a 30Khz

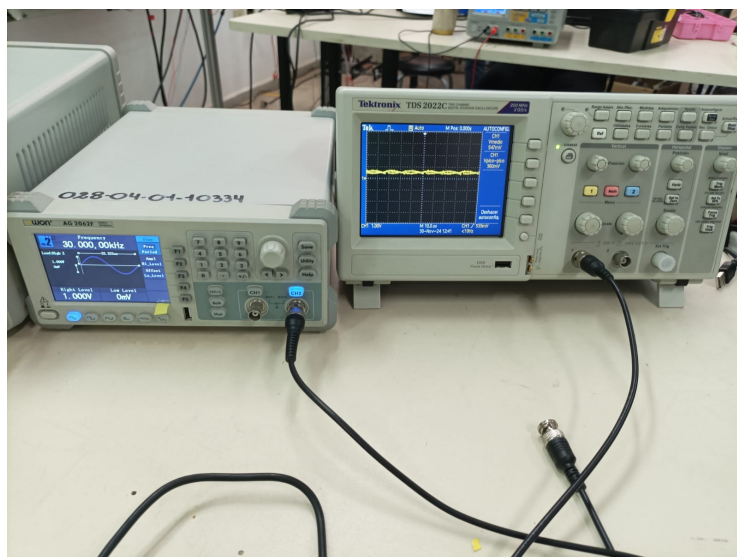


Figura 21: Prueba con una señal de 30kHz.

En la Figura 20 se observa que, para una señal de entrada de 1 Vpp a 20 kHz, la señal de salida presenta un valor de 740 mV, lo cual se aproxima al valor teórico de 707 mV. Este resultado valida el correcto funcionamiento del filtro diseñado, ya que comienza a aplicar la atenuación prevista en la frecuencia de corte (20 kHz). A partir de dicha frecuencia, el

filtro restringe progresivamente el paso de señales, como se evidencia en la Figura 21. Por otro lado, las señales de entrada con frecuencias inferiores a la frecuencia de corte son transmitidas sin atenuación significativa, tal como se muestra en las Figuras 18 y 19. En cuanto al filtro de reconstrucción, se observó que la salida obtenida fue idéntica a la del filtro antialias, razón por la cual no se incluyen las gráficas correspondientes a este filtro.

8. Conclusión

En este proyecto se logró la implementación exitosa de varios efectos de audio, como el vibrato, chorus, flanger y eco, utilizando el microcontrolador PSoC-5LP. Los algoritmos diseñados permiten ajustar parámetros clave de los efectos, como los tiempos de retardo y las modulaciones, lo que facilita la verificación y la evaluación auditiva de su funcionamiento. A través de diversas pruebas y ajustes, se confirmó que los efectos operan de acuerdo con lo esperado, proporcionando una experiencia sonora coherente y de calidad, además de ser fácilmente manipulables.

El microcontrolador PSoC-5LP ha sido una plataforma valiosa para este proyecto, gracias a su flexibilidad y la facilidad de integración de diferentes funciones. La interfaz intuitiva de PSoC Creator y su capacidad para manejar eficientemente los recursos del sistema fueron esenciales para el desarrollo y la prueba de los efectos. Esta herramienta no solo facilitó la creación de los algoritmos, sino que también permitió realizar ajustes y optimizaciones de manera rápida y sencilla, demostrando su idoneidad para proyectos de DSP.

En conclusión, los resultados obtenidos demuestran que, utilizando herramientas adecuadas como el microcontrolador PSoC-5LP, es posible implementar de manera eficiente y exitosa efectos de audio de calidad. La experiencia adquirida a lo largo del desarrollo de este proyecto resalta la importancia de contar con plataformas de desarrollo flexibles que permitan tanto la experimentación como la optimización de algoritmos. A pesar de que el proyecto se centró en efectos básicos, abre la puerta a futuras investigaciones y desarrollos en el campo del procesamiento digital de señales, donde se pueden explorar técnicas más avanzadas y nuevas posibilidades en la creación de efectos de audio.

Bibliografía

- [1] D. Marshall, *Audio Effects*, 2007. Available: https://users.cs.cf.ac.uk/Dave.Marshall/Multimedia/PDF/07_Audio_Effects.pdf.
- [2] eMastered, *Audio Effects Explained*, Available: <https://emastered.com/es/blog/audio-effects-explained>.
- [3] A. Adam, *Synthesis and Audio Effects*, Available: <https://sound.eti.pg.gda.pl/student/eim/synteza/adamx/eindex.html>.
- [4] Ariat-Tech, *LM324 Operational Amplifiers - Pinouts, Application Examples, Size Packages, Datasheets*, Available: <https://www.ariat-tech.com/blog/lm324-operational-amplifiers-pinouts,application-examples,size-packages,datasheets.html>.
- [5] El Cajón del Electrónico, *Filtro de Reconstrucción: Principios y Aplicaciones*, Available: <https://elcajondelelectronico.com/tag/filtro-de-reconstruccion/#:~:text=Filtro%20de%20reconstrucci%C3%B3n%3A%20Asegura%20que,la%20frecuencia%20m%C3%A1xima%20de%20trabajo..>